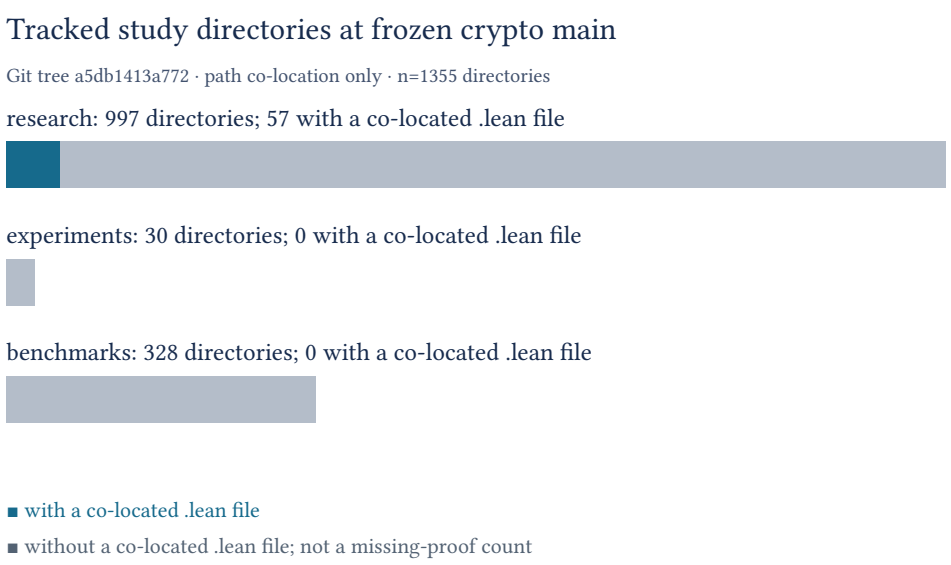
Crypto formal coverage: frozen path inventory

28 October 2026 · source commit `a5db1413a772ff2250aad38c5c05f6715647a6d5` · tree `d96fe62d14eb999972fa47aad6a920f333c97a66`

This is a syntactic navigation aid. It does not certify mathematical claims, Lean builds, experimental evidence, or code correctness. Its chart shows only where tracked Lean files sit relative to study directories.



Method and observations

`tools/formal_coverage_inventory.py` reads the complete frozen Git tree with `git ls-tree -r -z` and all tracked `.lean` blobs with `git cat-file --batch`. The 165,588 tracked blobs and one Git submodule entry are recorded separately. The submodule contents are not traversed. `inventory.json` records exact paths, blob IDs, and regex-derived theorem or lemma line pointers.

- 1355 study directories: 997 research, 30 experiments, 328 benchmarks.
- 242 `.lean` files in the repository; 229 under those study roots.
- 57 study directories with a co-located `.lean` file; 1298 without one.
- 4192 lexical declaration pointers. The regex can include commented text or omit other declaration forms.

Interpretation and next gate

Directory co-location is not a claim-to-theorem map. A benchmark may contain no theorem to prove; a theorem may live in a shared library or another repository. Review each substantive written claim against its exact Lean statement and hypotheses, run the pinned theorem build and axiom audit, then independently review the underlying experiment. Do not turn these path counts into a proof-coverage percentage.

Reproduction

From a checkout containing the frozen commit, run `python3 tools/formal_coverage_inventory.py a5db1413a772ff2250aad38c5c05f6715647a6d5 docs/reports/2026-10-28-formal-coverage/inventory.json`, then `python3 docs/reports/2026-10-28-formal-coverage/render.py`. This metadata script does not run cryptographic research or performance measurements.